

Encapsulamiento y abstracción

Material de lectura sobre el tema: Complementar lo presentado aquí con:

- Sebesta: Capítulo 10
- Scott: Capítulo 10

Las abstracciones son una vista o representación de una entidad que incluye los atributos más significativos.

Un **tipo de dato abstracto** es un tipo de dato definido por el usuario que satisface las siguientes condiciones:

1. La representación de los objetos del tipo son definidos como unidades sintácticas simples.
2. La representación del tipo es escondida del programa que utiliza estos objetos, por lo tanto las únicas operaciones posibles son aquellas provistas por la definición del tipo.

El concepto central detrás del diseño de abstracciones definidas por el programador es el de *ocultamiento de información*. Cuando la información es encapsulada en una abstracción, significa que el usuario de dicha abstracción:

- No necesita conocer la información oculta para poder hacer uso de la abstracción, y
- No está permitida la manipulación directa de la información oculta, aunque se desee hacerlo.

El **ocultamiento de información** es una cuestión de diseño de programas; es posible en cualquier programa que sea diseñado apropiadamente, independientemente del lenguaje de programación utilizado.

El **encapsulamiento** es una cuestión de diseño del lenguaje; una abstracción resulta efectivamente encapsulada solo cuando el lenguaje prohíbe el acceso a la información oculta en la abstracción.

El oscurecimiento es un objetivo de diseño de suma importancia para lograr un buen encapsulamiento. Para lograr estos objetivos, es necesario dividir el módulo en dos partes: interface e implementación.

Veamos el siguiente ejemplo en ADA:

```
package Stack_Pack is
  type stack_type is limited private;
  max_size: constant := 100;

  function empty(stk: in stack_type) return Boolean;
  procedure push(stk: in out stack_type; elem: in Integer);
  procedure pop(stk: in out stack_type);
  function top(stk: in stack_type) return Integer;
  private -- hidden from clients
  type list_type is array (1..max_size) of Integer;
  type stack_type is record
    list: list_type;
    topsub: Integer range 0..max_size := 0;
  end record;
  .....
end Stack_Pack
```

Diseño de tipos de datos abstractos

- Unidad sintáctica para definir el tipo de dato abstracto.
- Operaciones built-in
 - Asignaciones
 - Comparación
 - Operaciones comunes
 - Iteradores
 - Constructores
 - Destruyores
- Tipo de datos abstractos parametrizados.
- Extensiones de tipos: Subtipo vs. Herencia.

Implementación Orientada a objetos

Representación de clases

Es necesario mantener la siguiente información:

1. El nombre de la clase.
2. La superclase (que en general es *object* en caso de que no se especifique).
3. Los nombres de las variables de instancia.
4. Un diccionario con los métodos de clase.
5. Un diccionario con los métodos de instancia.

Registro de activación

Se mantiene un registro de activación, de manera tal que es posible mantener la información relevante a la activación de un método. El registro sigue manteniendo tres partes como en su versión para lenguajes imperativos:

- a. Entorno: contexto utilizado para la ejecución del método (entorno local y no local).
- b. Instrucciones: la instrucción a ejecutarse cuando el método se reanude.
- c. Emisor: el registro de activación del método que envió el mensaje invocando éste método (enlace dinámico).

A la hora de enviar un mensaje, debemos:

- a. Crear el registro de activación para el método llamado.
- b. Identificar el método invocado extrayendo el patrón del objeto receptor o superclase.
- c. Transmitir los parámetros al registro de activación del receptor.
- d. Suspender al llamador, salvando su estado en el registro de activación.
- e. Establecer el camino de regreso al emisor y colocar el registro de activación como el registro activo.

Al retornar, debemos:

- a. Transmitir el objeto resultante (si lo hay) al emisor.
- b. Reanudar la ejecución del emisor restaurando su estado con la información en su registro de activación.

Hay al menos dos partes de la programación orientada a objetos que provocan preguntas interesantes a los implementadores de lenguajes de programación.

- Las estructuras de almacenamiento para las variables de instancia.
- La ligadura dinámica de los mensajes a los métodos.

- Implementación de constructores Orientados a Objetos.
- Almacenamiento de los datos de una instancia: La estructura de almacenamiento tiene claras similitudes al registro. Se llama CIR (Class Instance Record).

La estructura del CIR es estática, se construye en tiempo de compilación y es utilizada como template para la creación de instancias de clase. Los accesos a los atributos de una instancia se resuelven de igual manera que los accesos a las componentes del registro y con la misma eficiencia.

¿Herencia?

Al considerar herencia es importante incluir algún mecanismo que permita resolver la vinculación dinámica de código. Las estrategias son las siguientes:

- Implementación basada en delegación.
- Implementación basada en copia.

Ambas estrategias tienen sus ventajas y desventajas. La primera hace un mejor uso de memoria pero en un claro detrimento del tiempo de ejecución (debido al costo de búsqueda implicado). La segunda utiliza más memoria pero las búsquedas son directas. De hecho, en un lenguaje compilado se reduce a accesos basados en desplazamientos dentro de la tabla de métodos (*vtable*).

Tengan en cuenta que las aproximaciones basadas en copia y delegación están pensadas solo para la tabla de métodos, no para los atributos que definen los objetos. La identidad de un objeto está dada por los valores de los atributos que lo definen (no importa si son heredados o “propios”). Teniendo en cuenta esto, pensar en una delegación de atributos es inconcebible.

Noten que estamos hablando de atributos de instancia, no de atributos de clase. Estos últimos son compartidos por todos los objetos instancia de esa clase.